Universidade de Brasília
Departamento de Ciência da Computação
Bacharelado em Ciência da Computação

Projeto do Banco de Registradores do RISC-V

Aluno: Lucas Vasconcelos Pirani Carneiro
Matrícula: 251031785

1 Introdução

O banco de registradores armazena temporariamente operandos e resultados utilizados pelas instruções, fornecendo acesso direto a esses valores durante a execução do processador [2]. Na arquitetura RISC-V, ele é composto por 32 registradores de 32 bits, identificados de `x0` a `x31`, com duas portas de leitura e uma porta de escrita síncrona. Os sinais `rs1` e `rs2` selecionam os registradores lidos, enquanto `rd`, `data` e `wren` definem, respectivamente, o registrador de destino, o dado a ser escrito e a habilitação da escrita. O registrador `x0` possui valor constantemente zero: escritas direcionadas a ele não alteram seu conteúdo e sua leitura retorna zero [2].

Este trabalho implementa, em VHDL, um banco de registradores compatível com a interface especificada para a atividade [1]. A implementação contempla 32 registradores de 32 bits, duas portas de leitura, uma escrita síncrona e o comportamento constante de `x0`. Um *testbench* verifica a escrita, as leituras e o registrador zero por meio de asserções.

2 Implementação

2.1 Interface

A entidade `xregs` implementa a interface do banco de registradores conforme prevista na especificação.

```
1  entity xregs is
2      generic (WSIZE : natural := 32);
3      port
4      (
5          clk, wren : in std_logic;
6          rs1, rs2, rd : in std_logic_vector(4 downto 0);
7          data : in std_logic_vector(WSIZE - 1 downto 0);
8          ro1, ro2 : out std_logic_vector(WSIZE - 1 downto 0)
9      );
10 end xregs;
```

Listing 1: Interface da entidade xregs.

O parâmetro genérico `WSIZE` define a largura dos registradores e assume o valor padrão de 32 bits. Os endereços `rs1`, `rs2` e `rd` possuem 5 bits, suficientes para selecionar um dos $2^5 = 32$ registradores. Os sinais `rs1` e `rs2` selecionam os registradores lidos, cujos valores são disponibilizados em `ro1` e `ro2`. Na escrita, `rd` seleciona o registrador de destino, `data` fornece o valor a ser armazenado e `wren` habilita a operação, sincronizada pelo sinal `clk`.

2.2 Arquitetura

O banco de registradores foi descrito comportamentalmente em VHDL. Os 32 registradores são representados pelo vetor `registers`, do tipo `reg_array`, cujas posições possuem largura

definida pelo parâmetro `WSIZE`. O vetor é inicializado com zeros e cada registrador é acessado por seu índice, de 0 a 31.

A escrita é síncrona: em cada borda de subida de `clk`, o valor de `data` é armazenado no registrador selecionado por `rd` apenas se `wren` estiver habilitado e `rd` for diferente de zero. Essa condição preserva o valor constante do registrador `x0`.

As leituras são combinacionais e realizadas por atribuições concorrentes. Os endereços `rs1` e `rs2` selecionam os valores apresentados em `ro1` e `ro2`, respectivamente. Quando um endereço corresponde a `x0`, a saída associada recebe zero; caso contrário, recebe o conteúdo do registrador selecionado. Assim, a implementação disponibiliza simultaneamente dois operandos e realiza a escrita de acordo com o clock e o sinal de controle.

O código completo da ULA está disponível no arquivo `xregs.vhd`.

3 Testes e Testbench

O funcionamento do banco de registradores foi verificado por meio de um *testbench* em VHDL, implementado no arquivo `xregs_tb.vhd`. O *testbench* instancia o componente `xregs`, gera o clock e aplica diferentes combinações de entrada. As saídas são comparadas aos valores esperados por comandos `assert`, que reportam erros durante a simulação.

3.1 Casos de teste

1. **Escrita no registrador `x0`:** com `wren` habilitado, `rd` igual a zero e `data` igual a `DEADBEEF`, verifica-se, após a borda de subida do clock, que `ro1` e `ro2` permanecem iguais a zero.

2. **Preenchimento dos registradores `x1` a `x31`:** cada registrador recebe o valor $100i+7$, em que i é seu endereço. Por exemplo, `x1` recebe 107, `x2` recebe 207 e `x31` recebe 3107.

3. **Escrita desabilitada:** com `wren` desabilitado, tenta-se escrever `DEADBEEF` em `x5`. A leitura desse registrador deve retornar 507, confirmando que seu conteúdo não foi alterado.

4. **Duas portas de leitura:** `rs1` percorre os endereços de 1 a 31, enquanto `rs2` percorre de 31 a 1. Para cada combinação, as saídas `ro1` e `ro2` são comparadas aos valores armazenados nos registradores selecionados.

5. **Verificação final de `x0`:** com `rs1` e `rs2` iguais a zero, verifica-se que ambas as saídas retornam zero após as operações anteriores.

3.2 Implementação da constante zero

A constante zero do registrador `XREGS[0]` foi implementada por meio de lógica condicional na descrição VHDL. Na escrita, o circuito verifica se o endereço de destino é diferente de `00000`,

descartando qualquer tentativa de escrita em `x0`. Na leitura, as saídas `ro1` e `ro2` são diretamente atribuídas a zero quando seus respectivos endereços selecionam esse registrador.

Portanto, a opção utilizada foi implementar o comportamento de `x0` por meio de condições lógicas na escrita e nas saídas de leitura, em vez de permitir que seu conteúdo fosse alterado como o dos demais registradores.

3.3 Simulação

A simulação foi executada na plataforma EDA Playground com o simulador `GHDL 5.1.1`. Ao final da execução, o testbench apresentou a mensagem de conclusão dos testes e não foram observadas mensagens de falha das asserções.

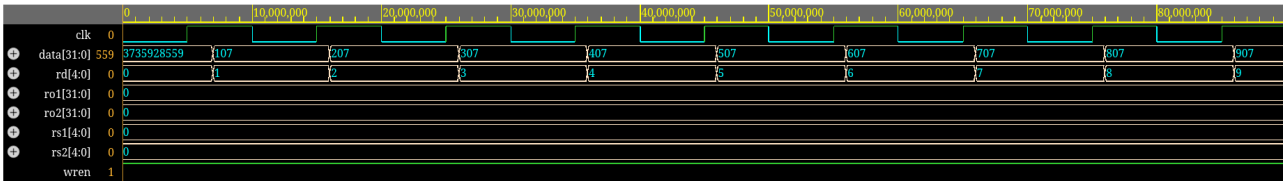


Figura 1: Formas de onda da primeira parte.

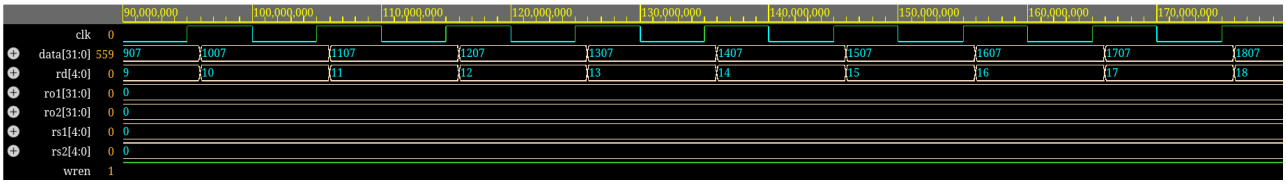


Figura 2: Formas de onda da segunda parte.

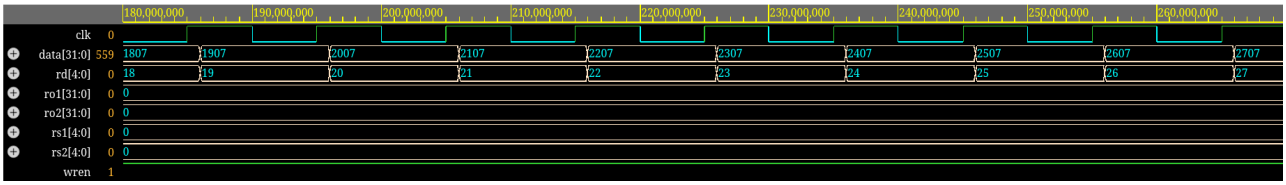


Figura 3: Formas de onda da terceira parte.

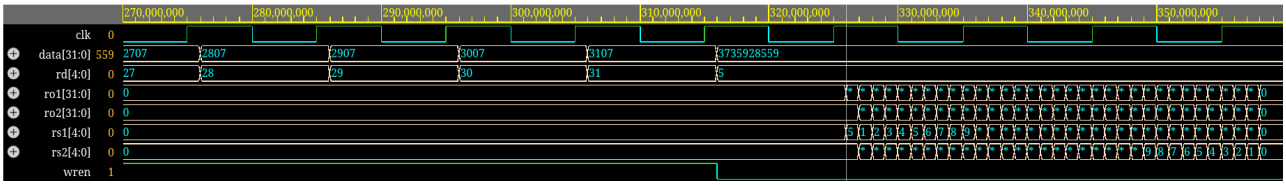


Figura 4: Formas de onda da quarta parte.

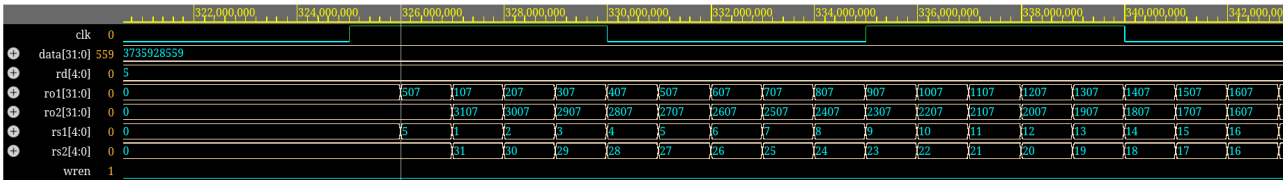


Figura 5: Formas de onda da quinta parte.

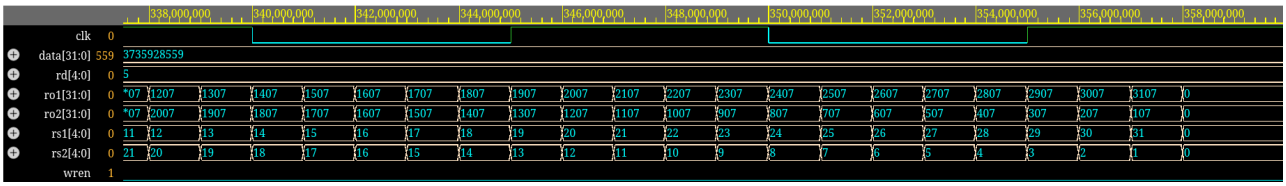


Figura 6: Formas de onda da sexta parte.

4 Conclusão

Este trabalho implementou, em VHDL, um banco de registradores para a arquitetura RISC-V, conforme a interface e os requisitos da especificação da atividade. O componente possui 32 registradores de 32 bits, duas portas de leitura combinacionais e uma porta de escrita síncrona.

A escrita ocorre na borda de subida do clock quando `wren` está habilitado e o endereço de destino é diferente de zero. O registrador `x0` mantém valor constante igual a zero: escritas para esse endereço são ignoradas e sua seleção retorna zero nas saídas de leitura.

Referências

[1] Especificação do trabalho: *Projeto do Banco de Registradores do RISC-V*. Material fornecido pelo(a) professor(a), s.d.

[2] PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. 2. ed. Morgan Kaufmann, 2020. ISBN: 978-0-12-820331-6.